

Investigação Operacional
2017/2018

- Aulas Práticas -

Introdução

ao

Matlab

1. O sistema Matlab

O Matlab, cujo nome deriva de **matrix laboratory**, é um sistema interactivo cujo elemento de dados básico é um **array** que não necessita de dimensões.

Permite resolver muitos problemas computacionais de carácter técnico, sobretudo os que envolvem formulações vectoriais e matriciais, numa fracção do tempo que seria necessário para escrever um programa equivalente numa linguagem como, por exemplo, C.

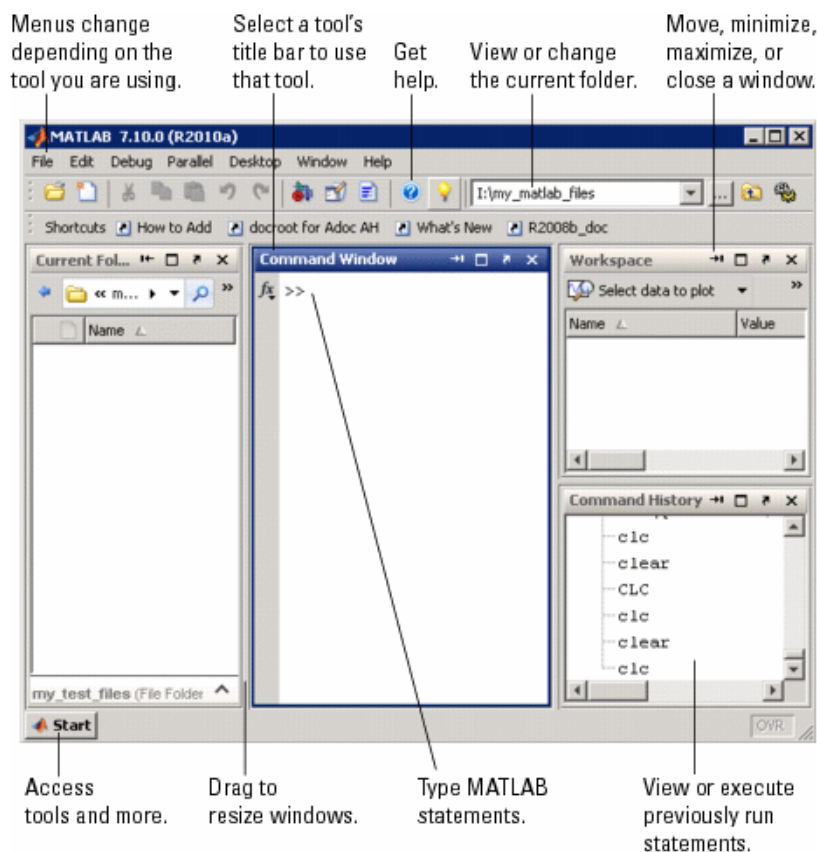
O Matlab consiste num pacote de funções orientadas fundamentalmente para o cálculo científico. Apresenta-se sob a forma de um **conjunto de bibliotecas (toolboxes)**, cada uma compreendendo funções específicas.

Exemplos:

- Neural Networks Toolbox
- Optimization Toolbox,
- Fuzzy Toolbox

1.1 O ambiente de trabalho do Matlab

Quando se inicia o Matlab entra-se no ambiente de trabalho desta aplicação, o qual dá acesso a um conjunto de ferramentas que permitem gerir ficheiros, variáveis e aplicações associadas com o Matlab.

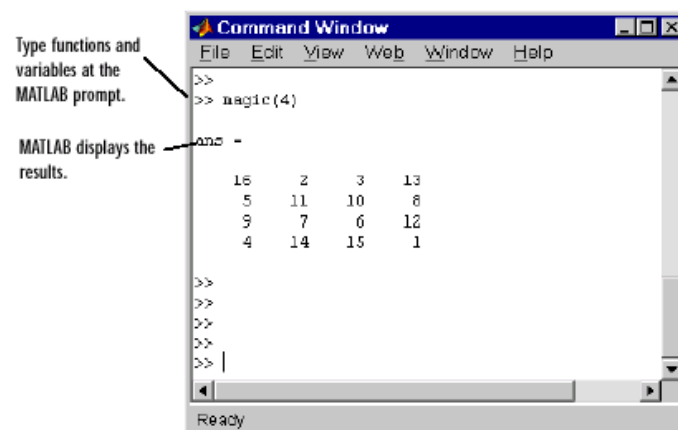


O acesso às diversas ferramentas que o Matlab disponibiliza pode fazer-se através do *desktop*.

1.2 Principais ferramentas disponíveis

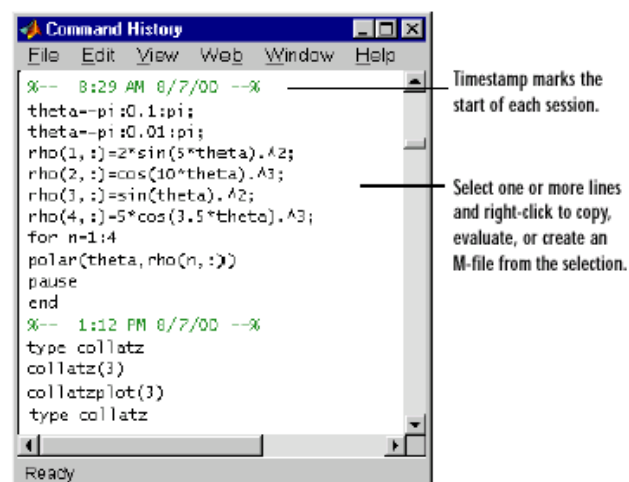
➤ *Command Window (Janela de Comandos)*

A *Janela de Comandos* permite introduzir variáveis e executar funções ou ficheiros *.m (M-Files).



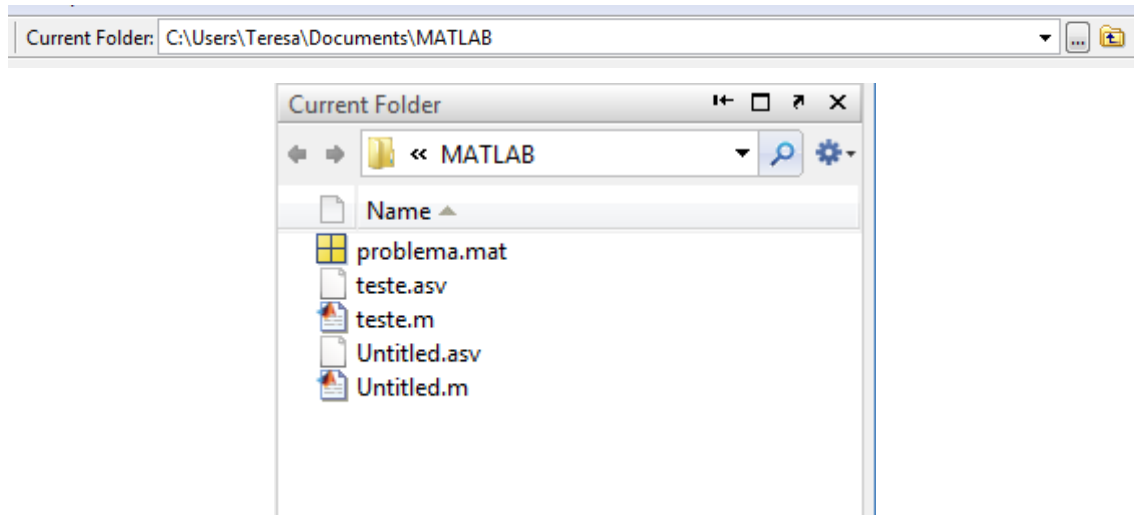
➤ *Command History (Histórico de Comandos)*

Todas as linhas introduzidas na *Janela de Comandos* são incluídas na janela correspondente ao *Histórico de Comandos*. Nessa janela podem ver-se as funções anteriormente executadas, bem como copiar e executar linhas seleccionadas.



➤ **Current Folder**

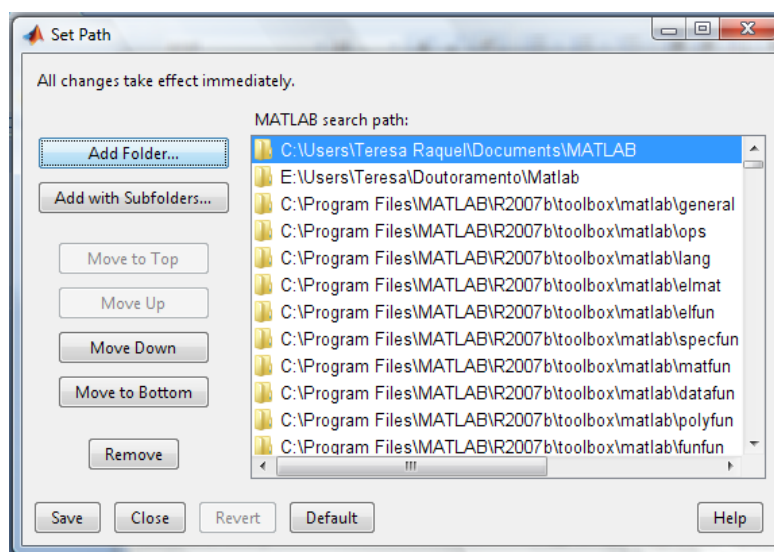
Uma forma muito rápida de visualizar ou de alterar a pasta actual, consiste em aceder ao campo *Current Folder* existente na barra de ferramentas do ambiente de trabalho, como se mostra na figura seguinte.



Search Path

Qualquer ficheiro que se pretenda executar em Matlab deve residir ou na pasta actual (*current folder*) ou numa pasta que faça parte do *search path*. Por defeito, todos os ficheiros fornecidos com as *toolboxes* do Matlab se encontram incluídos no *search path*.

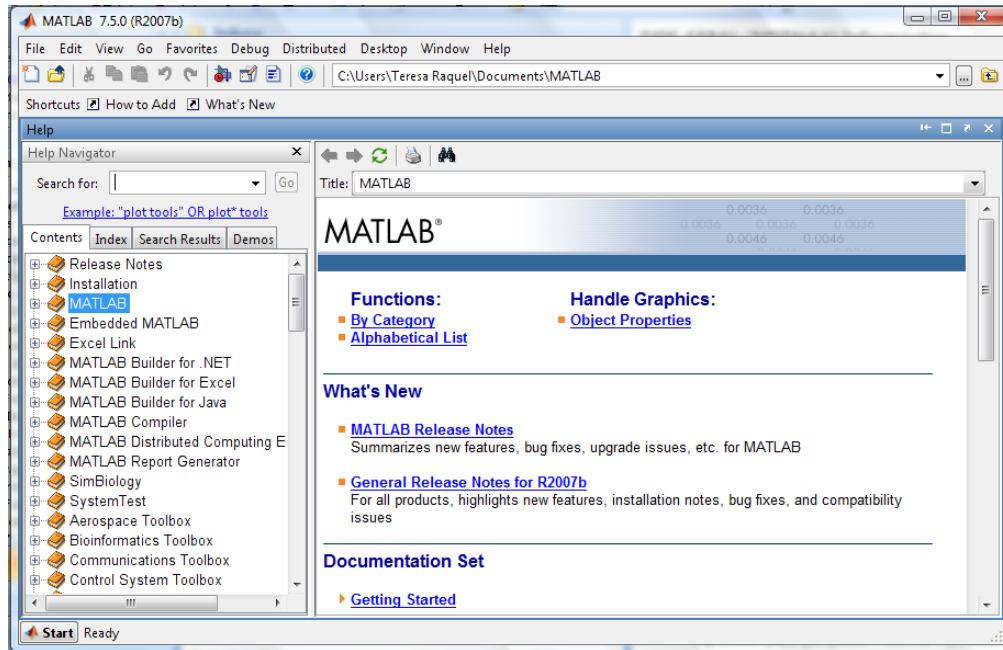
Para verificar quais as pastas que fazem parte do *search path*, ou para o alterar, deve ser seleccionada a opção **Set Path** do menu **File** e usada a caixa de diálogo respectiva.



➤ **Help Browser**

Esta ferramenta permite pesquisar e visualizar documentação relativa a todos os produtos da MathWorks. O *Help Browser* é um *Web Browser* integrado no ambiente de trabalho do Matlab.

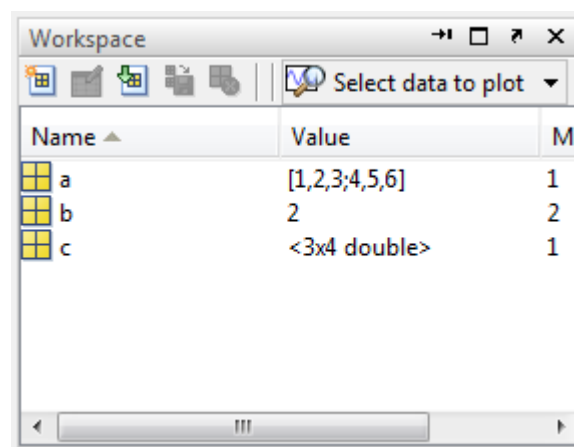
O acesso ao *Help Browser* faz-se de várias formas, como por exemplo, através do menu, ou da introdução do comando *helpbrowser* na *Janela de Comandos*.



➤ **Workspace Browser**

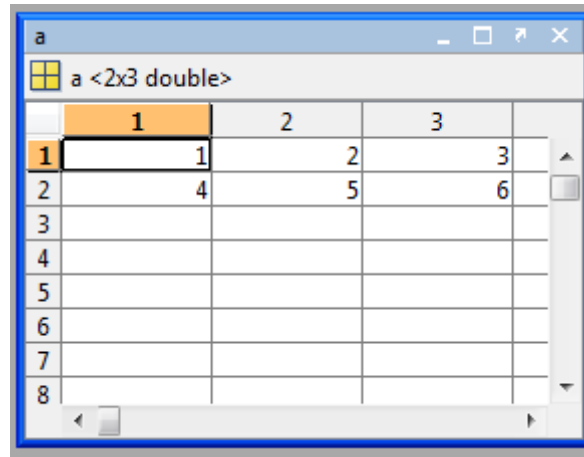
O espaço de trabalho (*workspace*) do Matlab consiste num conjunto de variáveis criadas durante uma sessão de trabalho e armazenadas em memória.

Para visualizar o *workspace* e a informação sobre cada uma das variáveis, utiliza-se o *Workspace Browser* ou, alternativamente, as funções *who* e *whos*.



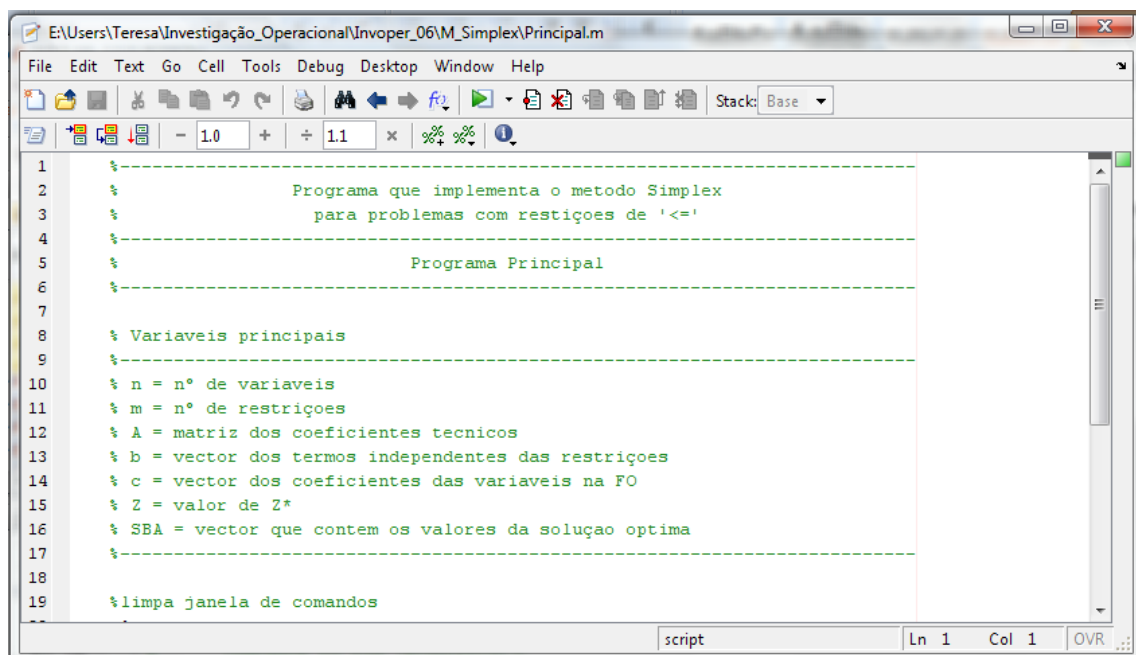
Array Editor

O *Array Editor* surge quando se carrega duas vezes sobre uma determinada variável do *workspace*. Este editor permite visualizar/alterar os *arrays* pertencentes ao *workspace*.



➤ Editor/Debugger

Utiliza-se o *Editor/Debugger* para criar ficheiros *.m, os quais contêm programas cujo objectivo é executar funções do Matlab. O *Editor/Debugger* proporciona um interface gráfico que permite, para além da edição básica de texto, o *debugging* de ficheiros *.m.



2. Definição de matrizes e operações sobre matrizes

Como se referiu anteriormente, em Matlab o elemento básico é a matriz.

2.1 Introdução / definição de matrizes

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

2.1.1 Introdução de forma directa ou explícita

```
>> A=[1 2 3;4 5 6]
A =
     1     2     3
     4     5     6
```

👉 **Nota:** O “;” funciona como operador de mudança de linha. Em alternativa é possível definir uma matriz premindo a tecla “ENTER” sempre que se pretenda efectuar uma mudança de linha na matriz.

2.1.2 Definição com base no resultado de uma operação

```
>> C=2*A
C =
     2     4     6
     8    10    12
```

2.2 Aceder a elementos de matrizes

2.2.1 Aceder a apenas um elemento

$A(i,j)$ – Permite o acesso ao elemento situado na linha i , coluna j .

```
>> res = A (1,2)
res = 2
```

2.2.2 Aceder a um conjunto de elementos

$A(i_{\min}:i_{\max}, j_{\min}:j_{\max})$

```
>> res = A(1:2,2:3)
res =
     2     3
     5     6
```

Se usar simplesmente o operador ':' sem indicar quaisquer valores limite, é equivalente a seleccionar 'todas as linhas' ou 'todas as colunas'

```
>> res = A(1,:)
res =
    1     2     3
```

2.3 Operações elementares

$$B = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \end{bmatrix}$$

```
>> B=[0 1 2;3 4 5]
B =
    0     1     2
    3     4     5
```

- Adição de matrizes:

```
>> A+B
ans =
    1     3     5
    7     9    11
```

- Transposta de uma matriz:

```
>> A'
ans =
    1     4
    2     5
    3     6
```

- Multiplicação de matrizes:

```
>> A*B
??? Error using ==> *
Inner matrix dimensions must agree.

>> D=A'
D =
    1     4
    2     5
    3     6

>> B*D
ans =
    8    17
   26    62
```


- Multiplicação de matrizes elemento a elemento (usa-se o operador '.' antes do operador '*'):

```
>> A.*B
ans =
    0    2    6
   12   20   30
```

- Inversa de uma matriz:

```
>> C=[4 1 9; 4 5 6; 7 8 9]
C =
    4    1    9
    4    5    6
    7    8    9

>> inv(C)
ans =
    0.0909   -1.9091    1.1818
   -0.1818    0.8182   -0.3636
    0.0909    0.7576   -0.4848
```

- Diagonal principal de uma matriz:

```
>> diag(C)
ans =
    4
    5
    9
```

- Matriz de uns (*ones(m,n): m=nº de linhas; n=nº de colunas*):

```
>> ones(3,4)
ans =
    1    1    1    1
    1    1    1    1
    1    1    1    1
```

- Matriz de zeros (*zeros(m,n): m=nº de linhas; n=nº de colunas*):

```
>> zeros(2,5)
ans =
    0    0    0    0    0
    0    0    0    0    0
```

- Matriz Identidade (*eye(m)* $m=n^{\circ}$ de linhas= n° de colunas)

```
>> eye(3)
```

```
ans =
```

```
1  0  0
0  1  0
0  0  1
```

2.4 Outras funções

- Valor absoluto: **abs(X)**

Calcula o valor absoluto dos elementos de X.

```
>> M=[-1 2 -3; 0 -8 4];
```

```
>> abs(M)
```

```
ans =
```

```
1  2  3
0  8  4
```

- Valor médio: **mean(X)**

Se X for um vector, devolve o valor médio dos elementos de X;

Se X for uma matriz, devolve um vector linha em que cada elemento é o valor médio de cada coluna de X.

```
>> V=[ 7  3  1];
```

```
>> mean(V)
```

```
ans =
```

```
3.6667
```

```
>> mean(M)
```

```
ans =
```

```
-0.5000 -3.0000  0.5000
```

```
% Para obter o mínimo da matriz:
```

```
>> mean(mean(M))
```

```
ans =
```

```
-1
```

- Valor máximo / mínimo: **max(X) / min(X)**

Se X for um vector, devolve o valor máximo / mínimo dos elementos de X;

Se X for uma matriz, devolve um vector linha em que cada elemento é o valor máximo / mínimo de cada coluna de X.

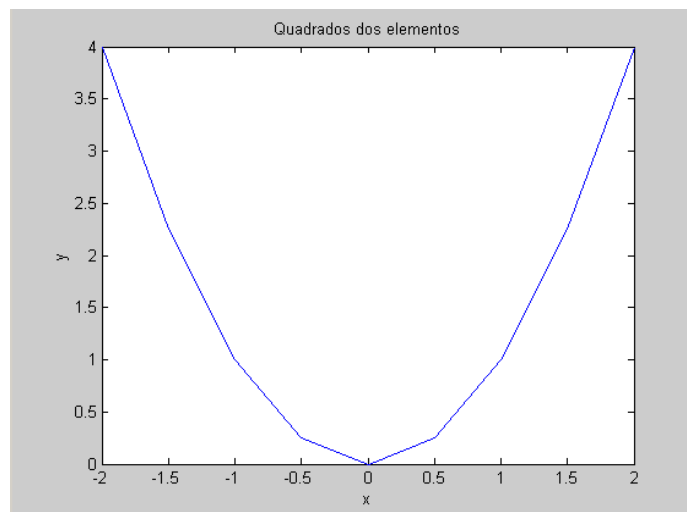
- ...

3. Representação gráfica de resultados

- Gráfico 2 D **plot(x,y)**
- Legenda no eixo das abcissas **xlabel('...')**
- Legenda no eixo das ordenadas **ylabel('...')**
- Título do gráfico **title('...')**
- ...

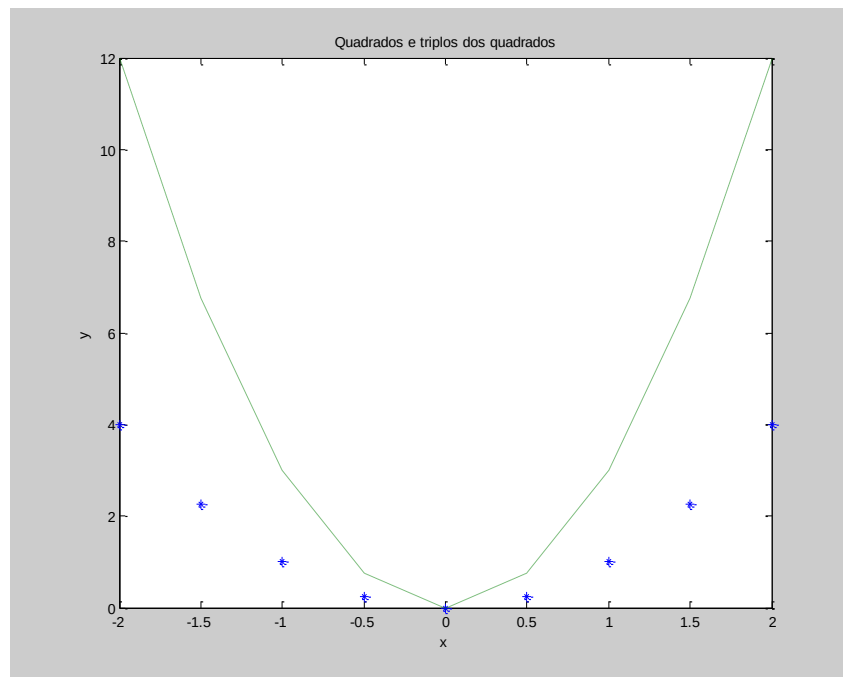
3.1 Representação gráfica de uma função

```
%      y = x^2
x = -2:0.5:2;
y = x.^2;
plot (x,y)
title('Quadrados dos elementos');
xlabel ('x')
ylabel ('y')
```



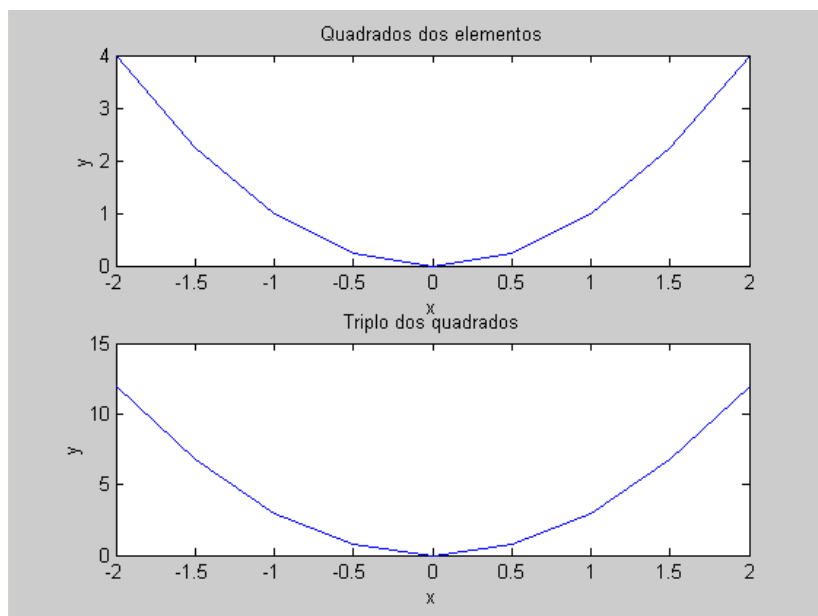
3.2 Representação de duas ou mais funções no mesmo gráfico

```
%      y = x^2 ; y1=3*y
x = -2:0.5:2;
y = x.^2;
y1 = 3*y;
plot (x, y, '*', x, y1, ':')
title('Quadrados e triplos dos quadrados')
xlabel('x')
ylabel('y')
```



3.3 Visualização simultânea de vários gráficos

```
x = -2:0.5:2;  
y = x.^2;  
y1=3*y;  
subplot(2,1,1);  
plot(x,y)  
title('Quadrados dos elementos')  
xlabel('x');  
ylabel('y');  
subplot(2,1,2)  
plot(x,y1)  
grid  
title('Triplo dos quadrados')  
xlabel('x');  
ylabel('y');
```



4. Ficheiros *.m (M-files)

Os ficheiros que contêm código em linguagem do Matlab são designados por ficheiros *.m ou *M-files*. Podem criar-se estes ficheiros com um editor de texto e depois usá-los como qualquer outra função ou comando do Matlab.

Existem dois tipos de ficheiros *.m:

- **Scripts** – não aceitam argumentos de entrada nem retornam quaisquer argumentos de saída. Operam com dados do *workspace*.
- **Funções** – podem aceitar quer argumentos de entrada, quer argumentos de saída. As variáveis internas são locais à função.

Uma vez efectuada a chamada de um ficheiro *.m na linha de comandos, o respectivo conjunto de instruções é executado sequencialmente.

5. Expressões Lógicas

- | | |
|----------------------|----|
| • Menor que | < |
| • Maior que | > |
| • Menor ou igual que | <= |
| • Maior ou igual que | >= |
| • Igual a | == |
| • Diferente de | ~= |
| • E lógico | && |
| • OU lógico | |
| • Negação | ~ |

6. Estruturas de controlo de fluxo

- **IF:** *executa instruções condicionalmente*

O formato geral é o seguinte:

```
if expressão
    instruções
elseif expressão
    instruções
else
    instruções
end
```

Exemplo:

```
if I == J
    A(I,J) = 2;
elseif abs(I-J) == 1
    A(I,J) = -1;
else
    A(I,J) = 0;
end
```

- **FOR:** *repete instruções um determinado número de vezes*

O formato geral é o seguinte:

```
for variável = expressão
    instrução
    ...
    Instrução
end
```

Exemplo:

```
for R = 1:N
    for C = 1:N
        A(R,C) = 1/(R+C-1);
    end
end
```

- **WHILE:** *repete instruções um determinado número de vezes*

O formato geral é o seguinte:

```
while expressão
    instrução
    ...
    Instrução
end
```

Exemplo:

```
R=1;
C=1;
while R<=N
    while C<=N
        A(R,C) = 1/(R+C-1);
        C=C+1;
    end
    R=R+1;
end
```

Exercício:

Construa um pequeno programa que conte os números positivos e negativos presentes numa matriz à escolha do utilizador.

7. Funções

Em Matlab o nome que identifica a função é o nome do ficheiro *.m que a contém. Nesta perspectiva, **deve ser sempre atribuído** à função o mesmo nome do ficheiro *.m. Se esse ficheiro contiver mais do que uma função, apenas a primeira pode ser chamada da janela de comandos ou por outro ficheiro *.m.

```
function [parametros_de_saida] = nome_funcao (parametros_entrada)
...
...
    Cálculo dos parâmetros de saída
...

```

Exercício:

Transforme o programa anteriormente desenvolvido numa função. Esta deve ter como parâmetro de entrada uma matriz e, como parâmetros de saída, o total de números positivos e de números negativos dessa matriz.

8. Ficheiros *.mat

O Matlab permite armazenar dados de uma determinada sessão para posterior utilização através de um ficheiro *.mat.

- | | |
|----------------------------------|--|
| • Armazenar variáveis | save <i>nome_ficheiro variáveis</i> |
| • Carregar variáveis armazenadas | load <i>nome_ficheiro variáveis</i> |
| • Identificar variáveis activas | who ou whos |
| • Limpar <i>workspace</i> | clear |

Exercício:

- Definir um conjunto de variáveis à escolha
- Verificar quais as variáveis que constituem o *workspace* (activas)
- Criar ficheiro “*variaveis.mat*”
- Apagar todas as variáveis definidas
- Carregar o ficheiro anteriormente criado
- Verificar *workspace*

O carregamento de um ficheiro *.mat, constitui uma forma alternativa de definição do *workspace* (conjunto das variáveis).